

Graph

```
class Graph:
    def __init__(self):
        self.graph = {}

    def add_vertex(self, vertex):
        if vertex not in self.graph:
            self.graph[vertex] = []

    def add_edge(self, vertex1, vertex2):
        if vertex1 in self.graph and vertex2 in self.graph:
            self.graph[vertex1].append(vertex2)
            self.graph[vertex2].append(vertex1)

    def display(self):
        for vertex in self.graph:
            print(f"{vertex} -> {self.graph[vertex]}")

g=Graph()
g.add_vertex(1)
g.add_vertex(2)
g.add_vertex(3)
g.add_vertex(4)
g.add_vertex(5)

g.add_edge(5,3)
g.add_edge(3,1)
g.add_edge(3,4)
g.add_edge(4,2)
g.add_edge(1,2)
g.add_edge(2,6)

g.display()
```

BFS

```

from collections import deque
def bfs(graph, start):
    visited = set()
    queue = deque([start])

    while queue:
        vertex = queue.popleft()
        if vertex not in visited:
            print(vertex, end=" ") # Process the vertex (e.g., print it)
            visited.add(vertex)
            queue.extend(neighbor for neighbor in graph[vertex] if neighbor not
in visited)

graph = {
    'A': ['O', 'D'],
    'O': ['A', 'M', 'U'],
    'D': ['S', 'A'],
    'M': ['O'],
    'U': ['O'],
    'S': ['D']
}
bfs(graph, 'A')

```

DFS

```

def dfs(graph, start):
    visited = set()
    stack = [start]

    while stack:
        vertex = stack.pop()

        if vertex not in visited:
            print(vertex, end=" ")
            visited.add(vertex)

            # add neighbors (reverse for correct order)
            stack.extend(reversed(graph[vertex]))

```

Greedy Algorithm

```

import heapq
def best_first_search(graph, start, goal, heuristic):
    visited = set()
    open_list = [(heuristic[start], start)]
    while open_list:
        _, current = heapq.heappop(open_list)
        if current == goal:
            print("Goal found:", current)
            return True
        if current not in visited:
            visited.add(current)
            print("Visiting:", current)
            for neighbor in graph[current]:
                if neighbor not in visited:
                    heapq.heappush(open_list, (heuristic[neighbor], neighbor))

# Example usage
graph = {
    "S": ['A', 'B'],
    "A": ['S', 'C', 'D'],
    "B": ['S', 'E', 'F'],
    "C": ['A'],
    "D": ['A'],
    "F": ['I', 'G', 'B'],
    "E": ['B', 'H'],
    "G": ['F'],
    "H": ['E'],
    "I": ['F']
}

heuristic = {
    "A": 12,
    "B": 4,
    "C": 7,
    "D": 3,
    "E": 8,
    "F": 2,
    "G": 0,
    "H": 4,
    "I": 9,
    "S": 13
}

best_first_search(graph, 'S', 'G', heuristic)

```

A *

```
import heapq

def astar(graph, start, goal, heuristic, cost):

    pq = [(heuristic[start], 0, start)]

    while pq:

        f, g, current = heapq.heappop(pq)

        print("Visiting:", current)

        if current == goal:
            print("Goal Found")
            return

        for neighbor in graph[current]:

            if (current, neighbor) in cost:

                new_g = g + cost[(current, neighbor)]

                new_f = new_g + heuristic[neighbor]

                heapq.heappush(
                    pq,
                    (new_f, new_g, neighbor)
                )

graph = {
    "A": ['B', 'C'],
    "B": ['A', 'D', 'D'],
    "C": ['A', 'E'],
    "D": ['B', 'F', 'G'],
    "E": ['C', 'G'],
    "F": ['D', 'G'],
    "G": ['D', 'E', 'F']
}

heuristic = {
```

```
"A": 5,  
"B": 6,  
"C": 4,  
"D": 3,  
"E": 3,  
"F": 1,  
"G": 0  
}  
  
cost = {  
    ("A", "B"): 1,  
    ("A", "C"): 4,  
    ("B", "D"): 3,  
    ("C", "E"): 5,  
    ("D", "F"): 2,  
    ("D", "G"): 4,  
    ("E", "G"): 3,  
    ("F", "G"): 1  
}  
  
astar(graph, 'A', 'G', heuristic, cost)
```